

TYPOGUARD

Technical Architecture & Development Handover Document

Project: TypoGuard

Version: 0.3 Architecture Plan

Current Extension Version: 0.1.0

Repository: `github.com/ukeme02/typoguard`

Branch: `main`

Architecture: Chrome Extension — Manifest V3 + Vanilla JavaScript

Storage: `chrome.storage.local`

Status: Active Development

Last Updated: September 2, 2026

1. EXECUTIVE SUMMARY

TypoGuard is a privacy-first browser extension designed to protect users from losing long-form text when a browser tab is accidentally closed, a page crashes, a web application refreshes, a session expires, or another unexpected event causes entered text to disappear.

The core purpose is simple:

TypoGuard should give users a safe way to recover text they have already typed when something goes wrong.

The extension is intentionally designed around privacy, safety, minimal permissions, and explicit user control.

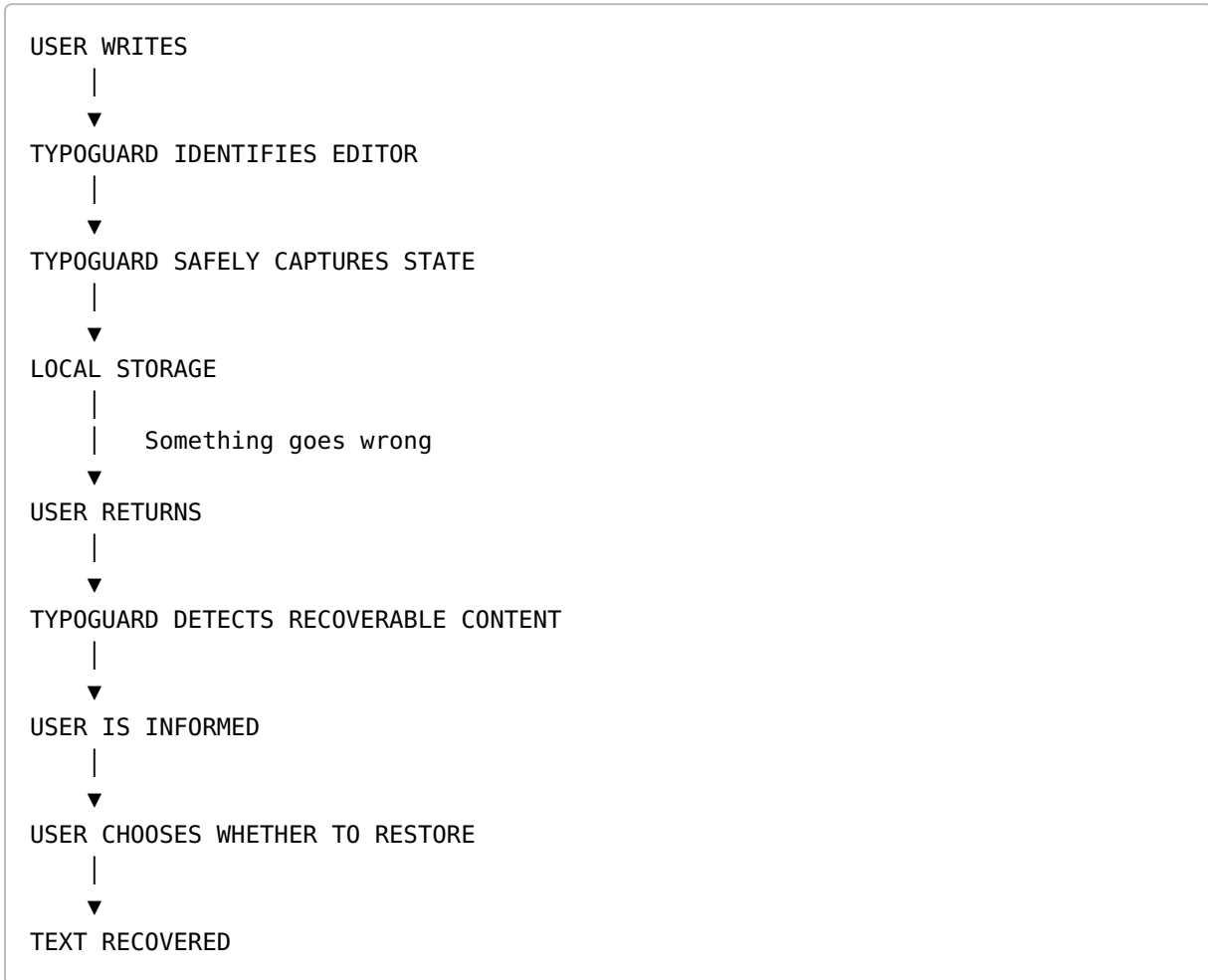
TypoGuard is **not** intended to be:

- A keylogger
- A general clipboard monitor
- A screen recorder
- A cloud-based writing backup service
- A network-based analytics system
- A tool that automatically submits or restores text without user consent

Instead, TypoGuard operates locally within the browser and focuses specifically on supported text-editing surfaces.

2. PRODUCT VISION

The long-term product experience is:



The critical design principle is:

TypoGuard should protect the user's work without taking control away from the user.

3. CORE DESIGN PRINCIPLES

3.1 Privacy First

User-entered content can be highly sensitive.

Therefore:

- Data remains local.
 - No remote text storage.
 - No unnecessary network communication.
 - No analytics service receiving editor contents.
 - No external API required for the core recovery system.
-

3.2 No Global Keyboard Capture

TypoGuard must not behave like a keylogger.

It should not globally monitor:

```
keydown  
keyup  
keypress
```

across the browser.

Instead, it observes supported editor elements and their state.

3.3 Limited Editor Scope

TypoGuard should focus on:

- `<textarea>`
- Appropriate text-editing inputs
- `contenteditable` elements

It should not indiscriminately capture:

- Password fields
 - Payment-card fields
 - Authentication fields
 - OTP fields
 - Search boxes
 - Short utility inputs
 - Other obviously sensitive or irrelevant fields
-

3.4 Explicit Restoration

TypoGuard must never silently replace existing user content.

Restoration should be:

```
graph TD; A[Detect recovery] --> B[Inform user]; B --> C[User chooses Restore]; C --> D[Restore content];
```

Not:

```
graph TD; A[Detect recovery] --> B[Automatically overwrite editor];
```

3.5 Local-First Architecture

The initial architecture deliberately avoids a backend.

This means the core system does not depend on:

- Servers
- Accounts
- APIs
- Cloud databases
- Authentication systems
- Internet connectivity

4. TECHNOLOGY STACK

Current Stack

Layer	Technology
Browser platform	Google Chrome

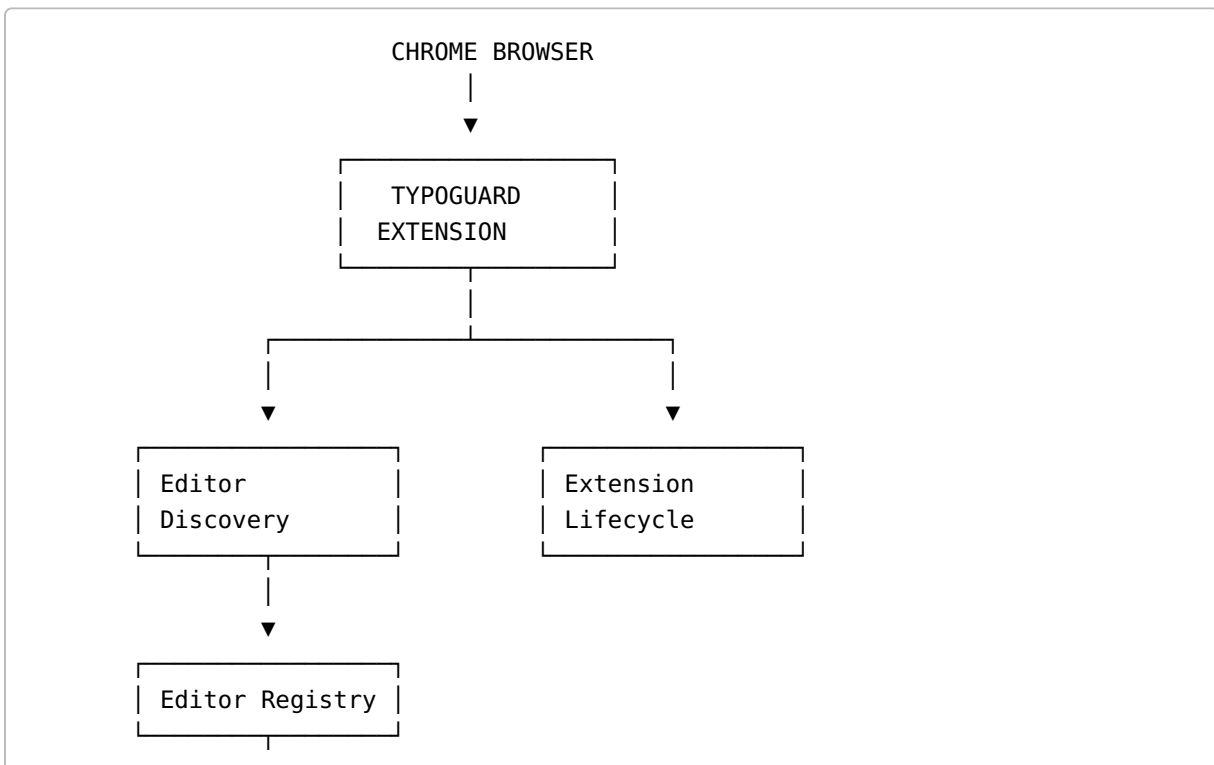
Layer	Technology
Extension standard	Manifest V3
Language	JavaScript
UI	Vanilla JavaScript / HTML / CSS
Storage	<code>chrome.storage.local</code>
Build system	None required initially
Framework	None
Repository	Git + GitHub
Development environment	VS Code + Linux/ChromeOS terminal

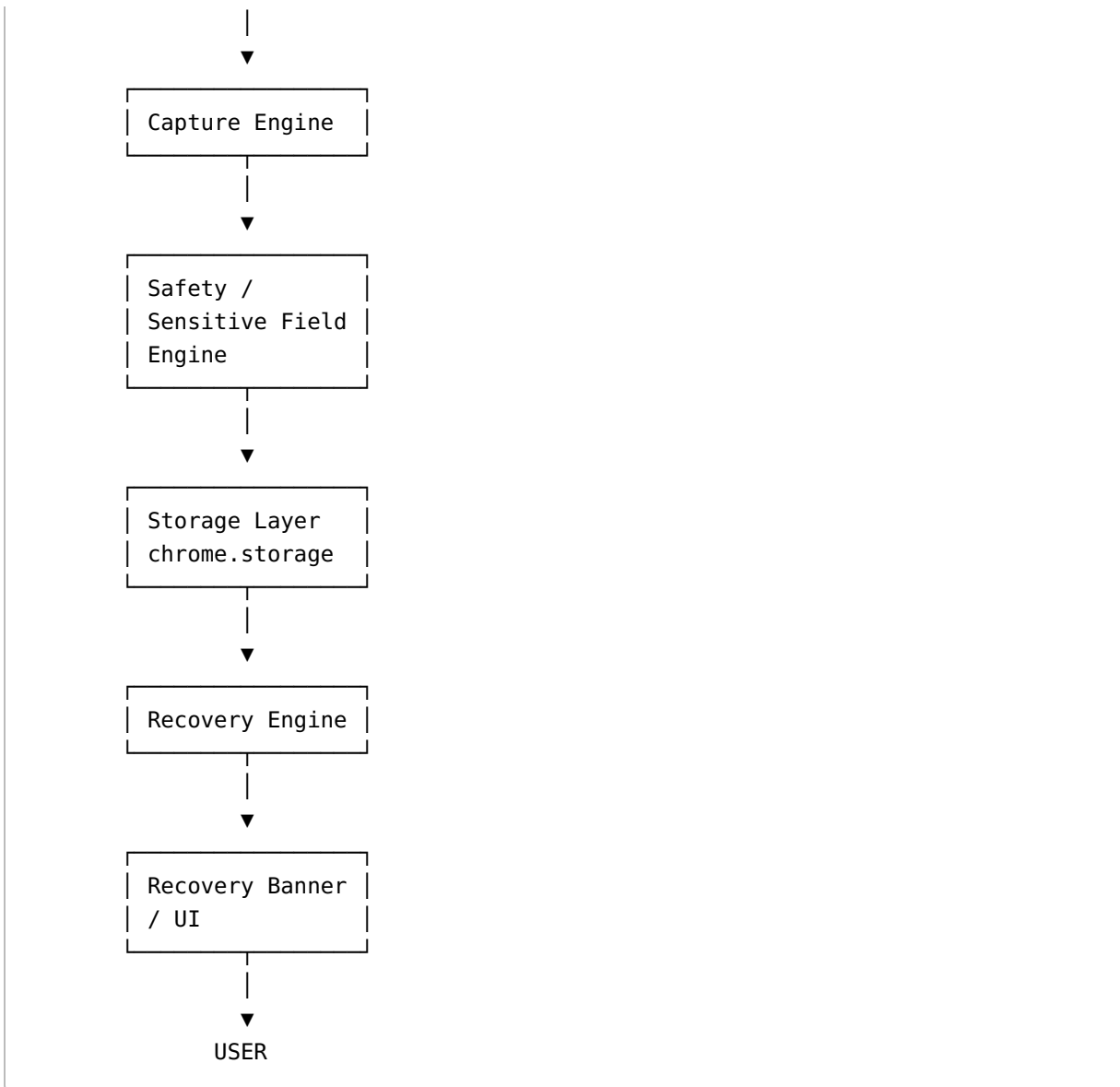
The project intentionally begins without unnecessary frameworks.

The architecture can evolve later if there is a compelling reason, but the recovery engine should remain lightweight and dependency-conscious.

5. HIGH-LEVEL SYSTEM ARCHITECTURE

The planned architecture is:





6. MAJOR SYSTEM COMPONENTS

6.1 Content Script

The content script operates within webpages and provides TypoGuard with access to supported editor elements.

Current file:

```
content.js
```

Current implementation:

```
console.log("TypoGuard: content script loaded.");
```

This is intentionally minimal at the current stage.

Its purpose at Phase 1 was simply to prove that Chrome can successfully inject and execute TypoGuard's content script.

7. EDITOR DISCOVERY

Planned Phase: 2

The Editor Discovery system answers:

"What editable areas currently exist on this page?"

It will identify supported editors such as:

```
<textarea></textarea>
```

and:

```
<div contenteditable="true"></div>
```

It will also handle qualifying text inputs where appropriate.

The discovery layer must distinguish useful editors from irrelevant fields.

Requirements

The system should:

- Detect editors already present.
- Detect dynamically created editors.
- Detect editors removed from the DOM.
- Avoid duplicate registration.
- Assign internal identities to editors.

- Maintain an editor registry.
- Avoid capturing content at discovery time.

The distinction between **discovering an editor** and **capturing its contents** is important.

Discovery should not automatically mean storage.

8. EDITOR REGISTRY

The Editor Registry provides a central representation of editors currently being monitored.

Conceptually:

Editor Registry

Editor A

- └ element reference
- └ internal ID
- └ type
- └ site/context information
- └ active state
- └ capture state

Editor B

- └ element reference
- └ internal ID
- └ type
- └ site/context information
- └ active state
- └ capture state

The registry allows downstream systems to interact with editors consistently without repeatedly scanning the entire DOM.

9. CAPTURE ENGINE

Planned Phase: 3

The Capture Engine is responsible for determining:

"When should TypoGuard save the current editor state?"

The system should not necessarily write to storage on every individual keystroke.

Instead, it should use controlled capture behavior such as:

```
User types
  ↓
Change detected
  ↓
Debounce / capture scheduling
  ↓
Current editor state evaluated
  ↓
Safety checks
  ↓
Snapshot generated
  ↓
Storage layer
```

The capture engine should eventually consider:

- Debouncing
- Minimum content thresholds
- Meaningful changes
- Capture frequency
- Duplicate snapshots
- Storage limits
- Cleanup policies

10. SAFETY / SENSITIVE-FIELD ENGINE

Planned Phase: 4

This is one of the most important components in TypoGuard.

The system must minimize the possibility of storing information that users would reasonably expect to remain private.

Potential exclusion signals include:

```
<input type="password">
```

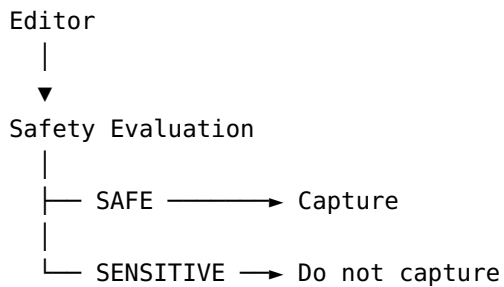
Sensitive autocomplete hints:

```
autocomplete="cc-number"
autocomplete="one-time-code"
```

Sensitive field names or purposes may also be considered.

The safety layer should operate before content reaches persistent storage.

Conceptually:



The safety system should follow a **fail-safe philosophy** where practical:

If an editor looks clearly sensitive, do not capture it.

11. STORAGE LAYER

Planned Phase: 5

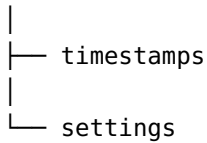
Storage will use:

```
chrome.storage.local
```

The initial product intentionally avoids cloud storage.

Conceptual structure:

```
TypoGuard Storage
├── editor records
├── snapshots
└── metadata
```



Stored records should contain enough information to support recovery without unnecessarily retaining additional personal data.

The storage architecture will eventually need:

- Versioning
- Size management
- Snapshot cleanup
- Retention rules
- Deduplication
- Recovery metadata
- Site/editor identification

12. END-TO-END RECOVERY

Planned Phase: 6

This phase connects the previously independent components.

Target flow:

```
Page
↓
Editor Discovery
↓
Editor Registry
↓
Safety Check
↓
Capture
↓
Local Storage
↓
Page/editor disappears
↓
Page returns
↓
Recovery candidate detected
↓
```

```
Recovery UI
↓
User chooses Restore
↓
Content restored
```

This is the first major milestone where TypoGuard becomes functionally useful.

13. RECOVERY BANNER / USER INTERFACE

Planned Phase: 7

When recoverable content exists, TypoGuard should communicate this clearly but unobtrusively.

Conceptual interface:

```
TypoGuard found text you may have lost.
[ Restore ]           [ Dismiss ]
```

The UI must prioritize:

- Clarity
- Trust
- Non-intrusiveness
- Explicit user control
- Accessibility

Restoration should always be deliberate.

14. SPA & DYNAMIC-PAGE HARDENING

Planned Phase: 8

Modern websites frequently behave as single-page applications.

Editors may appear after:

Initial page load

or disappear and reappear without a full navigation.

Examples include applications built with:

- React
- Vue
- Angular
- other dynamic DOM architectures

TypoGuard therefore cannot rely exclusively on initial page scanning.

The system will likely require DOM observation and lifecycle handling.

Conceptually:

```
Initial DOM
  ↓
Discover editors

DOM changes
  ↓
MutationObserver
  ↓
Discover new editors
  ↓
Remove stale editors
  ↓
Maintain registry
```

15. POPUP & HISTORY

Planned Phase: 9

The browser extension popup will eventually provide users with access to recovery-related functionality.

Potential capabilities:

- Recent recoverable drafts
- Recovery history
- Number of stored items

- Clear recovery data
- Extension status
- Quick access to settings

The popup must not expose unnecessary text content by default.

Privacy remains the primary design constraint.

16. SITE EXCLUSION & SETTINGS

Planned Phase: 10

Users should eventually be able to control where TypoGuard operates.

Potential settings:

```
TypoGuard
|
├─ Enable / Disable
├─ Excluded sites
├─ Retention period
├─ Maximum stored snapshots
└─ Recovery preferences
```

Example:

```
Never capture on:
- example-bank.com
- example-health.com
```

The user should always have a straightforward way to exclude a site.

17. SECURITY & PRIVACY AUDIT

Planned Phase: 11

Before broad testing or release, TypoGuard must undergo a dedicated security/privacy audit.

Areas include:

Permissions

Verify that the extension requests only what it actually needs.

Data

Verify that:

- Text is not transmitted.
- Text is not logged unnecessarily.
- Sensitive fields are excluded.
- Stored data is cleaned appropriately.

Code

Check for:

- Unsafe DOM operations
- Injection vulnerabilities
- Unexpected external requests
- Dependency risks
- Storage abuse

Extension behavior

Verify that TypoGuard does not:

- Capture passwords
- Capture globally typed data
- Monitor unrelated browser activity
- silently restore content
- send user content externally

18. COMPATIBILITY TESTING

Planned Phase: 12

The extension must be tested across different editor implementations.

Target categories include:

```
<textarea>  
contenteditable  
Rich text editors  
SPA editors
```

Dynamically generated editors
Long-form editors

Testing should cover:

- Normal typing
- Pasting
- Editing
- Deleting
- Navigation
- Refresh
- Tab closure/recovery scenarios
- Dynamic editor creation
- Multiple editors on one page

19. DOCUMENTATION

Planned Phase: 13

Documentation should eventually include:

README
Architecture
Installation
Development setup
Testing
Privacy model
Security model
Contribution guide
Issue reporting
Release process

The repository should allow another developer to clone the project and understand how the system works without requiring undocumented tribal knowledge.

20. CHROME WEB STORE PREPARATION

Planned Phase: 14

Before publication:

- Manifest reviewed
- Permissions minimized
- Store description prepared
- Privacy disclosures prepared
- Screenshots prepared
- Icons finalized
- Security review completed
- Testing completed
- Version number finalized

The extension should not be submitted until the privacy model and actual implementation agree.

21. RELEASE CANDIDATE

Planned Phase: 15

The final release candidate represents the point at which:

```
Architecture
+
Implementation
+
Testing
+
Security
+
Documentation
+
Store preparation
```

are all complete.

The release candidate should be treated as a controlled build rather than simply "the latest code."

22. DEVELOPMENT ROADMAP

Phase	Component	Status
0	Project setup & repository	● COMPLETE
1	Manifest & extension skeleton	● COMPLETE
2	Editor discovery	● NEXT
3	Capture engine	⌚
4	Safety / sensitive-field engine	⌚
5	Storage layer	⌚
6	End-to-end recovery	⌚
7	Recovery banner / UI	⌚
8	SPA & dynamic-page hardening	⌚
9	Popup & history	⌚
10	Site exclusion & settings	⌚
11	Security & privacy audit	⌚
12	Compatibility testing	⌚
13	Documentation	⌚
14	Chrome Web Store preparation	⌚
15	Release candidate	⌚

23. CURRENT PROJECT STATE

Phase 0 — COMPLETE

The development environment and repository were established.

Completed:

- Project directory created.
- Git repository initialized.
- Git identity configured.
- GitHub repository created.
- SSH authentication configured.

- Remote configured.
- `main` branch established.
- Initial repository pushed.

Initial commit:

```
76481f9
chore: initialize TypoGuard project
```

24. PHASE 1 — COMPLETE

The Chrome extension skeleton has been successfully established.

Current files:

```
TypoGuard/
|
├─ .git/
├─ .gitignore
├─ README.md
├─ content.js
└─ manifest.json
```

Current manifest:

```
{
  "manifest_version": 3,
  "name": "TypoGuard",
  "version": "0.1.0",
  "description": "Privacy-first browser extension for recovering long-form text
after accidental loss.",
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ]
}
```

Current content script:

```
console.log("TypoGuard: content script loaded.");
```

25. PHASE 1 VERIFICATION RESULTS

The extension was loaded successfully into Chrome.

Chrome confirmed:

```
TypoGuard  
Version: 0.1.0
```

Chrome assigned the extension ID:

```
iocjdmjplmonbekbhlgihpnnmlkbedhc
```

The content script was tested on a webpage.

Chrome DevTools successfully displayed:

```
TypoGuard: content script loaded.
```

This proves that:

```
Chrome  
↓  
Manifest  
↓  
Content Script  
↓  
Web Page
```

is functioning correctly.

26. CURRENT GIT STATE

Latest successful commit:

```
99a4084
feat: add initial extension skeleton
```

Successfully pushed to:

```
github.com/ukeme02/typoguard
```

Current repository state:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

This is an important project checkpoint.

The codebase is currently clean and synchronized with GitHub.

27. NEXT DEVELOPMENT TASK

Phase 2 — Editor Discovery

The immediate objective is to implement the first real TypoGuard subsystem.

The first milestone will be:

Detect supported editors without capturing their contents.

Initial test targets:

```
<textarea>
contenteditable="true"
```

The implementation will be introduced incrementally.

We will not immediately build the entire discovery engine.

The development sequence should be:

```
Create discovery module
↓
Detect textarea
↓
Test
↓
Detect contenteditable
↓
Test
↓
Filter irrelevant fields
↓
Test
↓
Handle dynamic editors
↓
Test
↓
Build editor registry
↓
Test
↓
Phase 2 acceptance
```

28. DEVELOPMENT CONTROL PROCESS

TypoGuard should be developed using controlled checkpoints.

For every phase:

1. Define

Determine exactly what the phase must accomplish.

2. Implement

Write only the code necessary for that milestone.

3. Test

Run deliberate tests against the intended behavior.

4. Inspect

Review the implementation for architectural or security problems.

5. Commit

Create a meaningful Git commit.

6. Push

Synchronize the repository with GitHub.

7. Update tracker

Record what was completed.

8. Proceed

Only then begin the next phase.

29. ENGINEERING RULE

A phase is **not complete because the code was written**.

A phase is complete only when its acceptance criteria have been demonstrated.

Therefore:

```
CODE ≠ COMPLETE

CODE
+
TEST
+
VERIFICATION
+
CLEAN REPOSITORY
=
COMPLETE
```

This rule should govern the entire TypoGuard development process.

30. ARCHITECTURAL PRINCIPLE FOR FUTURE CONTRIBUTORS

Any future contributor should ask the following before adding functionality:

Privacy

Does this feature unnecessarily expose or retain user content?

Security

Could this feature cause sensitive information to be captured?

Scope

Does this feature belong in TypoGuard's core recovery mission?

User control

Does this feature preserve explicit user choice?

Complexity

Can this be implemented more simply?

Compatibility

Will this work with modern dynamic web applications?

Maintainability

Will another developer understand this six months from now?

If the answer to these questions is not satisfactory, the feature should be reconsidered before implementation.

31. CURRENT ARCHITECTURAL MATURITY

At the end of Phase 1, TypoGuard is intentionally still a **skeleton**.

Implemented:


```
Chrome Extension
├──
└── Content Script
```

Planned:

```
Chrome Extension
├── Editor Discovery
├── Editor Registry
├── Capture Engine
├── Safety Engine
├── Storage Layer
├── Recovery Engine
├── Recovery UI
├── SPA Hardening
├── Popup
├── Settings
└── Security / Testing / Release
```

This is intentional.

We are building the system from the foundation upward rather than attempting to create the complete extension in one pass.

32. HANDOVER STATUS

● Completed

Phase 0 — Project Setup & Repository

Phase 1 — Manifest & Extension Skeleton

The repository is operational, Chrome accepts the extension, the content script executes, and GitHub is synchronized.

Immediate Next

Phase 2 — Editor Discovery

Remaining

Phases 3–15

All remaining functionality is currently planned but not yet implemented.

33. PROJECT NORTH STAR

The final TypoGuard experience should feel almost invisible when everything is working correctly.

The user should simply write.

If nothing goes wrong:

TypoGuard stays out of the way.

If something goes wrong:

TypoGuard is there.

If TypoGuard has something recoverable:

It tells the user.

If the user wants it:

The user's work comes back.

And throughout the entire process:

The user's data remains under the user's control.

CURRENT CHECKPOINT

Phase 0:  Complete

Phase 1:  Complete

Phase 2:  Ready to begin

Next engineering task: Build and test the Editor Discovery subsystem.